module3_architecture

Module 3 — Embedded Runtime Architecture & VHL System Design

Virtual Harmonium Lab · ESP32 Digital Musical Workstation

Engineering Architecture Document

Synthesized from: Module 1 (Instrument Architecture) + Module 2 (Playback Engine Architecture)
Date: 2026-07-27
Status: Authoritative Architecture Specification · Pre-Implementation

---

> Module 1: **"What information defines an instrument?"**
> Module 2: **"How is that information transformed into sound?"**
> Module 3: **"How do we build all of that on a microcontroller efficiently?"**

Table of Contents

1. Complete System Architecture
2. Separation of Responsibilities
3. VHL File Format Architecture
4. Builder Responsibilities
5. Runtime Responsibilities
6. Memory Architecture
7. Sample Storage Strategy
8. Runtime Data Structures
9. Lookup Tables
10. Voice Architecture
11. Timing Architecture
12. CPU Optimization Strategy
13. Scalability
14. Future Hardware Expansion
15. Source Asset Mapping

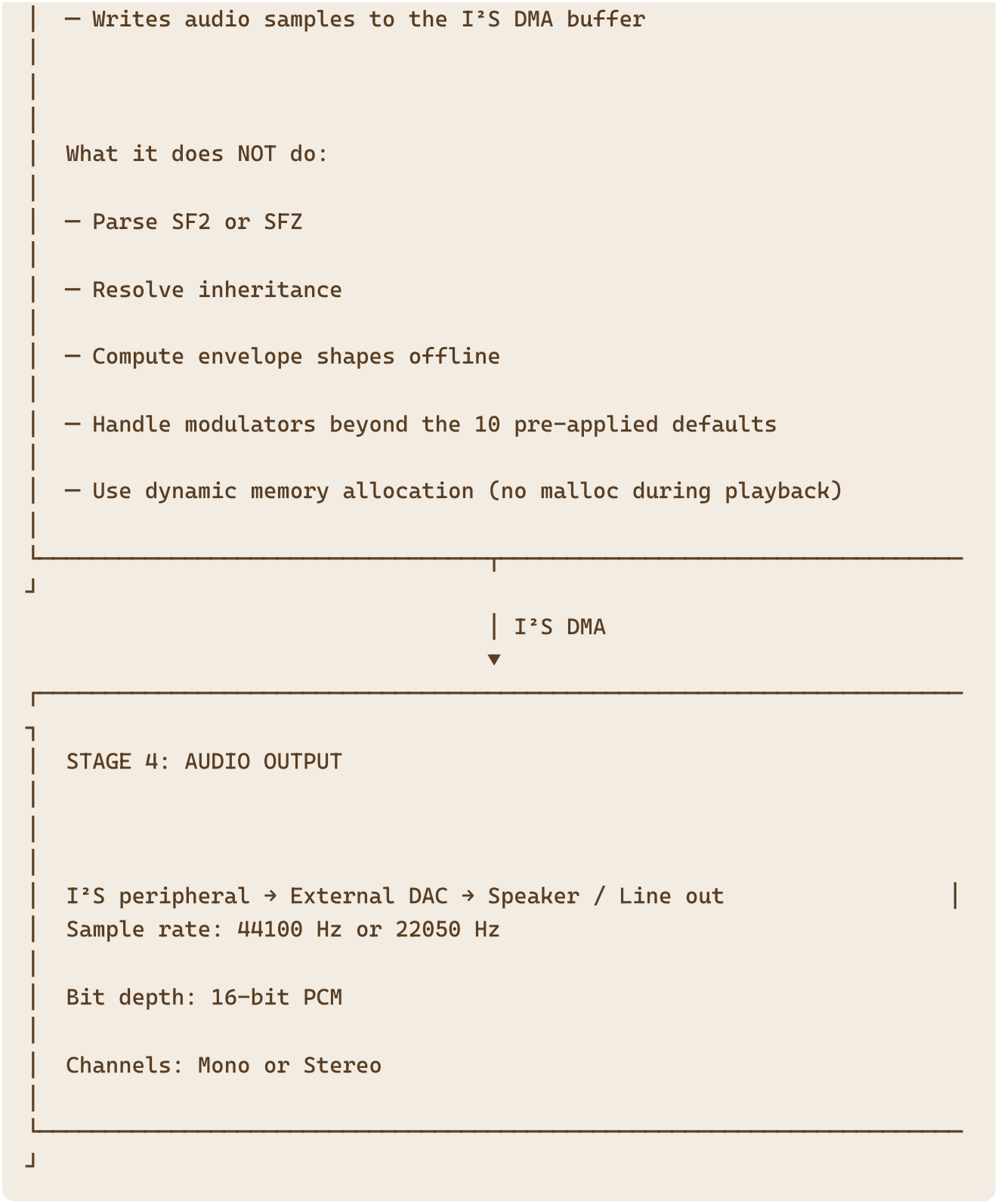
1. Complete System Architecture

The VHL Pipeline




An offline desktop tool (Python / C++ / Rust).
Never runs on the ESP32.
Never needs to be real-time.
Responsibilities:
— Parse SF2 / SFZ format
— Resolve SF2 preset→instrument→sample hierarchy
— Flatten global + zone parameters ("pre-bake")
— Apply and verify loop offsets
— Validate all parameter ranges
— Generate zone lookup tables
— Generate envelope coefficient tables
— Generate tuning tables
— Encode / compress sample data
— Assign contiguous memory addresses to samples
— Pack everything into a binary VHL package
— Generate metadata and version information
Output: .vhl package file
.vhl package
STAGE 2: VHL PACKAGE
(Binary File)

```
│
│
│
│  A self-describing binary file that contains:
│
│  — File header (magic bytes, version, checksum)
│
│  — Instrument manifest (names, IDs, preset numbers)
│
│  — Region table (pre-baked zone parameters, fully flat)
│
│  — Sample index (addresses, lengths, rates, root keys, loops)
│
│  — Lookup tables (key/vel → region list, tuning, envelope coeffs)
│
│  — Sample data (raw PCM, address-aligned for DMA)
│
│
│
│  This file is flashed to internal or external flash on the ESP32.
│
└──────────────────────────────────┬──────────────────────────────────┘
                                   │ flash / SD / PSRAM
                                   ▼
┌──────────────────────────────────────────────────────────────────────┐
│
│  STAGE 3: ESP32 RUNTIME                              (Embedded)
│
│
│
│  A lean, deterministic real-time engine that:
│
│  — Reads the VHL package at boot
│
│  — Loads the region table and sample index into RAM
│
│  — Optionally preloads sample data into PSRAM
│
│  — Handles MIDI Note-On / Note-Off events
│
│  — Performs region lookup (key × velocity → matching regions)
│
│  — Allocates voices from a pre-allocated static pool
│
│  — Runs per-block DSP: resampling, envelopes, LFOs, filter, mix
│
```



The Guiding Principle

The architecture is built on one central principle:

> **Every computation that can be done offline must be done offline.**

This means:

- The Builder is **arbitrarily complex** — it runs once on a desktop.
- The Runtime is **deliberately minimal** — it runs forever on a microcontroller.

- The VHL package is **self-contained and immediately usable** — no interpretation needed.

Embedded Notes — System Architecture

> **Why this pipeline matters for ESP32:** Every computation moved from Runtime to Builder is CPU time and RAM saved permanently. The ESP32 never needs to understand SF2 or SFZ. It only needs to understand the VHL format — which is designed to be maximally simple. This is the single most important architectural decision in the entire project.

---

2. Separation of Responsibilities

The Three Layers

```
┌─────────────────────────────────────────────────────────┐
│ EDITOR (Polyphone)                                      │
│                                                         │
│ Human-driven. Runs on desktop. No real-time constraint. │
│ Produces editable source assets (.sf2, .sfz).           │
├─────────────────────────────────────────────────────────┤
│ BUILDER (VHL Builder Tool)                              │
│                                                         │
│ Automated. Runs on desktop. No real-time constraint.    │
│ Consumes source assets. Produces the VHL package.       │
├─────────────────────────────────────────────────────────┤
│ RUNTIME (ESP32 Firmware)                                │
│                                                         │
│ Automated. Runs on ESP32. Hard real-time constraint.    │
│ Consumes the VHL package. Produces audio.               │
└─────────────────────────────────────────────────────────┘
```

Editor Responsibilities

The Editor (Polyphone) handles all human-facing creative tasks:

Responsibility	Reason
Sample recording and trimming	Requires human judgment
Loop point editing	Requires listening and visual alignment
Envelope parameter editing	Requires perceptual testing
Filter and LFO parameter editing	Requires listening

Responsibility	Reason
Key and velocity zone mapping	Requires musical knowledge
Preset definition	Requires musical knowledge
Modulator chain editing	Advanced SF2 configuration
Preview playback	Requires the full Polyphone engine

Rule: The Editor owns everything that requires a human to evaluate the result aurally.

Builder Responsibilities

The Builder handles all machine-transformable tasks that can be computed once and stored:

Responsibility	Reason
Parsing SF2/SFZ binary/text format	Format understanding, not needed at runtime
Resolving preset→instrument hierarchy	Two-stage lookup flattened to zero stages
Merging global + zone parameters	Inheritance removed; each zone is self-contained
Applying sample offset generators	Pre-computed loop/start/end absolute addresses
Validating parameter ranges	Catch errors before deployment
Resolving modulator defaults	10 default modulators pre-evaluated to modified parameters
Generating key×velocity lookup table	Zone search reduced to a table read
Computing envelope time→coefficient tables	`pow()` calls moved to build time
Computing tuning tables	`pow2(cents/1200)` moved to build time
Encoding samples (bit depth reduction)	Sample compression/truncation done once
Assigning contiguous sample addresses	DMA-aligned addresses for fast streaming
Computing sinc filter table (if used)	64KB table generated once at build time
Binary packing with checksums	Self-describing, validated package

Rule: The Builder owns everything that can be computed from the static instrument definition without knowledge of the played note.

Runtime Responsibilities

The Runtime handles only what is fundamentally note-dependent and time-dependent:

Responsibility	Reason
MIDI event reception and routing	Real-time, unpredictable timing
Region lookup (key × velocity → zone list)	Note-dependent — key/vel not known until played
Voice allocation from static pool	Dynamic state that changes with every note
Sample position tracking	Continuous per-sample state
Resampling (pitch playback ratio)	Note-dependent pitch ratio
Envelope phase advancement	Continuous per-block time-dependent state
LFO oscillation	Continuous per-block state
Filter coefficient update and application	Modulated by LFO/envelope, per-block
Volume coefficient interpolation	Per-sample smoothing
Voice mixing	Depends on how many voices are active
Voice stealing and polyphony management	Dynamic state
Exclusive class (choke) detection	Note-dependent
I²S DMA buffer writing	Real-time hardware interface

Rule: The Runtime owns only what is inherently dynamic: things that depend on which note was played, when it was played, or what time it is.

The Key Insight About Modulators

Module 2 revealed that the 10 SF2 default modulators have fixed sources, fixed amounts, and fixed destinations. Examples:

- Velocity → Attenuation: `attenuation += (127 - vel) * some_scale`
- Pitch Wheel → Pitch: `fineTune += bend * sensitivity`
- CC7 (Volume) → Attenuation: `attenuation += f(CC7)`

For a harmonium, **almost none of these are relevant at Note-On time**. The instrument is a fixed-tone reed organ; its volume is determined by the bellows (a physical controller), not by MIDI velocity in the traditional sense.

The Builder can pre-evaluate which default modulators are meaningful and bake their effects into the zone parameters directly where they are note-independent. Only the genuinely dynamic ones (pitch wheel, CC7 volume) need to remain in the Runtime.

Embedded Notes — Separation of Responsibilities

> **Why this separation is essential for ESP32:** Every task in the Builder column is computation that would otherwise happen either at boot (too slow) or at Note-On (too late, may cause latency spikes). By moving it to the Builder, the ESP32 boots in milliseconds and responds to Note-On within a few microseconds. This is the difference between a usable instrument and an unresponsive one.

---

3. VHL File Format Architecture

Design Philosophy

The VHL format is designed to be:

1. **Directly addressable** — the runtime can address any region or sample by a simple array index
2. **Self-contained** — no external dependencies at runtime
3. **Versioned** — allowing future format upgrades without breaking existing packages
4. **Validated** — checksums prevent corrupted packages from causing silent failures
5. **DMA-aligned** — sample data starts on a 4-byte (or 32-byte for PSRAM burst) boundary

Format Sections

```
┌─────────────────────────────────────────────────────────┐
│ SECTION 0: FILE HEADER                                  │
│                                                         │
│ — Magic bytes ("VHL1" or similar 4-byte signature)      │
│ — Format version (major.minor)                          │
│ — Total file size                                       │
```

```
| - CRC32 or SHA-256 of entire file                          |
| - Number of instruments                                    |
| - Offsets to each major section (absolute byte pos)        |
| - Build timestamp                                          |
| - Builder version                                          |
| - Target sample rate                                       |
| - Default polyphony                                        |
└────────────────────────────────────────────────────────────┘
┌────────────────────────────────────────────────────────────┐
| SECTION 1: INSTRUMENT MANIFEST                             |
|                                                            |
| - Array of instrument descriptors                          |
|   Each descriptor contains:                                |
|     - Instrument ID (integer)                              |
|     - Instrument name (null-terminated ASCII)              |
|     - Bank number + preset number (for MIDI mapping)       |
|     - Offset into Region Table for this instrument         |
|     - Number of regions for this instrument                |
|     - Offset into Lookup Table for this instrument         |
└────────────────────────────────────────────────────────────┘
┌────────────────────────────────────────────────────────────┐
| SECTION 2: REGION TABLE                                    |
|                                                            |
| - Flat array of fully pre-baked region descriptors         |
|   Each region descriptor contains:                         |
|     - key_lo, key_hi (uint8_t)                             |
|     - vel_lo, vel_hi (uint8_t)                             |
|     - exclusive_class (uint8_t, 0 = none)                  |
|     - loop_mode (uint8_t: 0/1/3)                           |
|     - sample_id (uint16_t: index into Sample Index)        |
|     - pan (int16_t: -500 to +500)                          |
|     - attenuation (uint16_t: in 0.1 dB units)              |
|     - vol_env: delay, attack, hold, decay,                 |
|                sustain, release (all in ms, uint16_t)      |
|     - mod_env: same 6 parameters                           |
|     - mod_env_to_pitch (int16_t: cents)                    |
|     - mod_env_to_filter_fc (int16_t: cents)                |
|     - filter_fc (uint16_t: in Hz)                          |
|     - filter_q (uint8_t: 0.1 dB units)                     |
|     - lfo_mod: delay, freq, to_pitch, to_filter,           |
|                to_volume (mix of int16_t/uint16_t)         |
|     - lfo_vib: delay, freq, to_pitch                       |
|     - scale_tuning (uint8_t: 0-100)                        |
|     - coarse_tune (int8_t: semitones)                      |
|     - fine_tune (int16_t: cents × 10)                      |
|     - key_hold_scale, key_decay_scale (int8_t)             |
|     - reverb_send, chorus_send (uint8_t: 0-100)            |
└────────────────────────────────────────────────────────────┘
| SECTION 3: SAMPLE INDEX                                    |
```

```
│                                                          │
│ — Flat array of sample descriptors                       │
│   Each descriptor contains:                              │
│     — sample_offset (uint32_t: byte offset in file)      │
│     — sample_length (uint32_t: total PCM frames)         │
│     — loop_start (uint32_t: PCM frame offset)            │
│     — loop_end (uint32_t: PCM frame offset)              │
│     — sample_rate (uint32_t: Hz, e.g. 44100)             │
│     — root_key (uint8_t: MIDI note 0–127)                │
│     — fine_tune (int8_t: cents, -50 to +50)              │
│     — bit_depth (uint8_t: 8 or 16)                       │
│     — channels (uint8_t: 1=mono, 2=stereo)               │
└──────────────────────────────────────────────────────────┘
┌──────────────────────────────────────────────────────────┐
│ SECTION 4: LOOKUP TABLES                                 │
│                                                          │
│ For each instrument:                                     │
│   A 128 × 128 table (key × velocity)                     │
│   Each cell = a small list of region_ids that match      │
│   that key + velocity combination.                       │
│                                                          │
│ Possible representation:                                 │
│   — Cell = offset + count into a sorted region list      │
│   — Or: direct region_id array for common keys           │
│                                                          │
│ Optional additional tables:                              │
│   — Tuning table: note → playback ratio (Q16.16)         │
│   — Envelope coeff tables: time → exp decay coeff        │
└──────────────────────────────────────────────────────────┘
┌──────────────────────────────────────────────────────────┐
│ SECTION 5: SAMPLE DATA                                   │
│                                                          │
│ — Raw PCM sample data, concatenated                      │
│ — Each sample starts at a 4-byte aligned address         │
│ — 16-bit signed PCM (most common target format)          │
│ — Padding between samples for alignment if needed        │
│ — This is the largest section (typically MBs)            │
└──────────────────────────────────────────────────────────┘
```

Versioning Strategy

The file header includes `format_version (uint16_t major, uint16_t minor)`. Rules:

- **Major version change**: incompatible format, runtime must reject
- **Minor version change**: additive change, runtime ignores unknown fields
- The runtime checks the major version at boot and halts if mismatched

The Flat Zone Design

The most important architectural decision in the VHL format is that **regions are fully flat**. There is no global zone. There is no instrument hierarchy. Every region contains its own complete set of parameters.

This means:

- The two-stage `Preset → Instrument` lookup (from Module 2) is eliminated entirely
- Global zone inheritance is resolved at build time
- Modulator default values are baked into the parameters
- The runtime never traverses a hierarchy — it accesses regions by index

Embedded Notes — File Format

> **Why flat regions matter for ESP32:** Polyphone's runtime performs two hierarchy traversals per Note-On (playPrst → playInst) plus inheritance resolution (global + zone). On ESP32, these traversals involve pointer chasing through a data model that doesn't exist in the VHL format. Flat regions mean: load region[i], read its parameters, done. No pointer chasing, no memory allocation, no conditional logic.
> The lookup table (Section 4) reduces region search from O(N) linear scan to O(1) table lookup. This is the second most important architectural decision.

---

4. Builder Responsibilities

Complete Builder Task List

The Builder is an offline desktop tool. It runs once per instrument update. It has no performance constraint. Its output must be correct and reproducible.

Phase 1: Parsing

- Parse the SF2 binary format (header, pdta chunks, smpl chunk)
- Parse the SFZ text format (global, group, region blocks)
- Validate all parsed data against the SF2 specification

Phase 2: Hierarchy Resolution

This is the core of the Builder's work. From Module 1, we know Polyphone's resolution order:

```
SF2: Sample default → Instrument global zone → Instrument zone → Preset
global zone → Preset zone
SFZ: Global block → Group block → Region block
```

The Builder resolves this hierarchy for every zone:

```
For each preset zone P that matches a given instrument I:
  For each instrument zone Z:
    Compute effective_param[i] for all 60 generators:
      = sample_default[i]        (always present)
      + inst_global_set[i]       (if set in global division)
      + inst_zone_set[i]         (if set in zone division)
      + prst_global_set[i] (add) (if set in global division — additive
for most params)
      + prst_zone_set[i]  (add)  (if set in zone division)

    Store result as a single flat VHL Region
```

This removes the two-stage lookup completely.

Phase 3: Default Modulator Pre-Application

From Module 2, the 10 SF2 default modulators include:

- Note-On velocity → Initial Attenuation (parabolic curve)
- Note-On velocity (inverse) → Filter Cutoff
- CC7 (Volume) → Initial Attenuation (linear)
- CC10 (Pan) → Pan
- CC11 (Expression) → Initial Attenuation (linear)
- CC91 (Reverb) → Reverb Send
- CC93 (Chorus) → Chorus Send
- Pitch Wheel → Fine Tune (via pitch wheel sensitivity)
- Pitch Wheel Sensitivity (RPN 0) → Fine Tune
- Poly Aftertouch → Initial Attenuation

Builder action: Identify which of these are note-independent (fixed at zone time) vs note-dependent:

- **Note-independent** (bake into region): CC7 default = 100, CC10 default = 64, CC11 default = 127, reverb/chorus defaults. These can be folded into the base parameter values.

- **Note-dependent** (keep in runtime): velocity→attenuation curve (velocity unknown until Note-On), pitch wheel (changes during playback).

The Builder folds note-independent modulator effects directly into the region parameters. The Runtime only needs to handle velocity→attenuation and pitch wheel.

Phase 4: Parameter Validation and Clamping

Validate every pre-baked parameter:

- Attack time ≥ 0, ≤ 100 seconds
- Loop start < Loop end < Sample length
- Root key 0–127
- Fine tune −100 to +100 cents
- etc.

If any parameter is out of range, clamp it and emit a warning. Never silently produce invalid regions.

Phase 5: Lookup Table Generation

Key × Velocity Zone Table

For each instrument, generate a 128×128 table where cell `[key][vel]` contains a list of region indices that match that key+velocity combination.

The full 128×128 = 16,384 entries would be expensive in RAM. Two alternative structures:

Option A: Dense region list per key

For each key (0–127), store a sorted list of (vel_lo, vel_hi, region_id) entries. At runtime, a simple binary search through a short list finds all matching regions.

Option B: Sparse hash map (key, vel_range) → region_ids

For instruments with few zones (like a harmonium with ~30 samples), a simple linear scan through the 30 regions is fast enough — 30 range comparisons at Note-On is negligible.

Recommended: For VHL v1, use a linear scan over the full region list. For VHL v2, add a key-indexed acceleration structure.

Tuning Table

For each semitone offset from root key, precompute:

```
tuning_ratio[d] = 2^(d / 12)   for d in -64..+63
```

Stored as Q16.16 fixed-point (16 integer bits, 16 fractional bits). This replaces the per-Note-On `pow2(cents/1200)` calculation with a table lookup.

Combined with the intra-semitone fine tune (−50 to +50 cents), which may still require a small interpolation, total pitch lookup becomes: table lookup + one small correction.

Envelope Coefficient Tables

From Module 2, the volume envelope decay formula is:

```
coef = pow(0.00001585 / (1 - sustain + 0.00001585), 1 / timeDecay)
```

This `pow()` call is expensive at runtime. The Builder precomputes a table:

```
envelope_decay_coef[time_ms] = pow(0.00001585, 1.0 / (time_ms / 1000.0
* chunkRate))
```

Indexed by time in milliseconds (0–10,000 ms → 10,001 entries of float32). At runtime, a table lookup + `sustain` level adjustment replaces the `pow()`.

Phase 6: Sample Encoding

- Convert 24-bit samples to 16-bit (with dithering if needed)
- Normalize amplitude to maximize dynamic range without clipping
- Optionally resample to the target sample rate (if different from source)
- Align each sample to a 4-byte boundary (insert padding if needed)
- Concatenate all samples into the sample data section

Phase 7: Binary Packing

- Write all sections in order with their absolute byte offsets in the header
- Compute and write the CRC32 or SHA-256 checksum of the entire file
- Write format version and build timestamp

Why Everything Possible Goes Into the Builder

Principle of deferred computation: Any computation that is not strictly dependent on Note-On time, current time, or current voice state belongs in the Builder.

This is not optional for embedded systems — it is mandatory. The ESP32 has roughly 20 microseconds to respond to a Note-On event before audible latency begins. Every microsecond saved in the Runtime is a microsecond that can be used for more voices, better quality resampling, or future effects.

Embedded Notes — Builder

> **For ESP32:** The Builder is the key enabler. Without it, the ESP32 would need to parse SF2 format (complex binary parser), resolve hierarchy (pointer traversal), run `pow()` at Note-On (expensive floating-point), and perform modulator graph resolution (iterative convergence algorithm). All of these are real-time nightmares. The Builder eliminates all of them.
> The Builder can be as slow and complex as needed — it runs on a developer's laptop, not on the device.

---

5. Runtime Responsibilities

Complete Runtime Task List

The Runtime only does what it must:

At Boot (Once)

- Read and validate the VHL file header (checksum, version)
- Load the Instrument Manifest into RAM
- Load the Region Table into RAM
- Load the Sample Index into RAM
- Load the Lookup Tables into RAM
- Optionally: preload sample data into PSRAM
- Pre-initialize the static Voice Pool (zero all voice state)
- Pre-initialize the static audio output buffers
- Configure I²S DMA at the target sample rate
- Register the DMA interrupt callback

On Note-On (Per Event)

- Receive MIDI note (channel, key, velocity)
- Map (channel, bank, program) → Instrument ID
- Apply velocity→attenuation curve (the one dynamic modulator)
- Scan the Region Table for matching zones (key × velocity)

- For each matching region:
  - Allocate one Voice from the Voice Pool
  - Populate the Voice with region parameters
  - Set the Voice's playback position to sample start
  - Initialize the Voice's Envelope state machine to Delay/Attack
  - Initialize the Voice's LFO state
  - Initialize the Voice's filter delay lines to 0
  - Set the Voice's *active* flag

On Note-Off (Per Event)

- Find all active Voices matching the (channel, key) pair
- Set each Voice's *release* flag → envelope enters Release phase
- Voices with `loop_mode = 3` exit their loop and play to end

On MIDI Pitch Wheel Change

- Update the shared pitch wheel delta (in cents)
- This is added to every active Voice's pitch calculation in the next audio block

On MIDI CC Change

- CC7 (Volume): update global volume attenuation
- CC10 (Pan): update the channel pan offset (added to voice pan in mixer)
- Ignore other CCs (or route to future effects chain)

Per DMA Interrupt (Per Audio Block)

This is the critical path — must complete before the DMA buffer empties.

```
On DMA callback:
  For each Voice in the Voice Pool:
    If voice is not active: skip
    Run Voice::processBlock(outputBuffer, blockSize)
  Mix all Voice outputs into the final stereo buffer
  Write final buffer to I²S DMA
```

Per Block, Per Voice

- Evaluate Volume Envelope: one value per chunk (256 samples)
- Evaluate Modulation Envelope: one value per chunk
- Evaluate Mod LFO: one value per chunk

- Evaluate Vib LFO: one value per chunk
- Compute pitch delta including LFO + mod envelope modulation
- Convert pitch delta to fixed-point resampling ratio
- Resample source samples into output buffer
- Compute filter coefficients (once per chunk, using mod envelope + LFO)
- Apply biquad filter (per sample)
- Apply volume envelope × conversion coefficient (per sample)
- Check Voice for completion (envelope = off, sample = end, no loop)
- Deactivate Voice if done

Per Block, Mixer

- For each active Voice: accumulate into L and R output buffers with pan
- Hard clip the output to [−1.0, +1.0]
- Convert float output to int16 for I²S DMA

What the Runtime Does NOT Do

Task	Why Eliminated
SF2 / SFZ parsing	Done by Builder
Hierarchy traversal	Flattened by Builder
Modulator graph resolution	Baked by Builder
`pow()` for envelope time conversion	Replaced by Builder lookup table
`log()` / `exp()` for pitch calculation	Replaced by Builder tuning table
Reverb / Chorus effects	Too expensive for v1
Dynamic memory allocation	Replaced by static Voice Pool
Multi-thread audio generation	Single ISR/callback model
Custom modulator support	Not needed for harmonium
Full MIDI channel management	Simplified to what harmonium needs

Embedded Notes — Runtime

> **For ESP32:** The Runtime must be deterministic. Every operation in the DMA callback must have a bounded worst-case execution time. Non-determinism (dynamic allocation, variable-length loops over unknown data) is fatal in hard real-time audio.
> The Region Table scan is O(N_regions) where N is small (30–100 for a harmonium). This is deterministic and fast.

> Everything that has a `pow()`, `log()`, `exp()` must either be eliminated or replaced with a lookup table in the Runtime.

---

6. Memory Architecture

ESP32 Memory Overview

Memory Type	Size	Characteristics
Internal SRAM (D-Cache)	320 KB	Fast, word-aligned access, CPU cache
Internal SRAM (IRAM)	128 KB	Used for code, fastest execution
Internal ROM	384 KB	Bootloader + WiFi/BT stack
External PSRAM (if present)	4–8 MB	Slower, 80 MHz SPI bus, cache-assisted
Internal Flash	4–16 MB	Very slow direct access, cached
External Flash (SPI)	16–128 MB	Accessible via SPI, slow streaming
SD Card	2–64 GB	Extremely slow, high latency, unreliable

RAM Partitioning

```
┌──────────────────────────────────────────────────────────┐
│ INTERNAL SRAM (320 KB total)                             │
│                                                          │
│ ├─ System / WiFi / BT stack       ~ 80 KB (reserved)     │
│ ├─ FreeRTOS / task stacks         ~ 20 KB                │
│ ├─ Runtime code globals / BSS     ~ 16 KB                │
│ ├─ Region Table (loaded at boot)  ~ 8 KB (100 regions)   │
│ ├─ Sample Index (loaded at boot)  ~ 4 KB (100 samples)   │
│ ├─ Lookup Tables (key/vel)        ~ 4 KB                 │
│ ├─ Voice Pool (32 voices)         ~ 16 KB (500B/voice)   │
│ ├─ DMA audio buffers (2 × 1KB)    ~ 2 KB (double-buffer) │
│ ├─ Mixer accumulation buffers     ~ 2 KB                 │
│ └─ Available headroom             ~ 168 KB               │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│ EXTERNAL PSRAM (4–8 MB, if available)                    │
│                                                          │
│ ├─ Preloaded sample data          ~ 2–6 MB               │
```

```
│  └─ Sinc filter tables (optional)  ~ 64 KB                  │
└─────────────────────────────────────────────────────────────┘


┌─────────────────────────────────────────────────────────────┐
│ INTERNAL FLASH (4–16 MB)                                    │
│                                                             │
│ ├─ Firmware (code + read-only data)   ~ 1–2 MB              │
│ ├─ VHL Package (small instrument)     ~ 1–8 MB              │
│ └─ Configuration / calibration data   ~ few KB              │
└─────────────────────────────────────────────────────────────┘
```

Voice Memory Allocation

Each Voice requires:

Field	Size
Playback position (uint32_t)	4 bytes
Fractional position (uint32_t)	4 bytes
Samples remaining until boundary (int32_t)	4 bytes
Region pointer or index (uint16_t)	2 bytes
Sample pointer (uint32_t)	4 bytes
Envelope state (8 bytes × 2 envelopes)	16 bytes
LFO state (12 bytes × 2 LFOs)	24 bytes
Filter delay lines (4 floats × 4 bytes)	16 bytes
Volume coefficient (float)	4 bytes
Vol/Mod/LFO per-chunk arrays (4 × 16 × float)	256 bytes
Flags (active, release, loop_mode)	4 bytes
Resampling scratch buffer (8 × int16_t)	16 bytes

Total per voice: ~354 bytes → budget 400 bytes per voice for alignment

32 voices × 400 bytes = **12.8 KB** — easily fits in internal SRAM.

DMA Buffer Design

Double-buffering is essential for glitch-free audio:

- **Buffer A**: being filled by the synthesis engine
- **Buffer B**: being read by the I²S DMA peripheral

When DMA interrupt fires (Buffer B empty):

1. Swap A and B
2. DMA begins reading new Buffer A
3. Synthesis engine begins filling new Buffer B

Buffer size: 512 samples × 2 channels × 2 bytes = **2 KB per buffer × 2 = 4 KB total**

Immutable Data

The following data is immutable after boot:

Data	Location	Reason
Region Table	RAM (copied from flash)	Never changes during playback
Sample Index	RAM (copied from flash)	Never changes
Lookup Tables	RAM (copied from flash)	Never changes
Sample PCM data	Flash or PSRAM	Read-only reference data
Sinc filter tables	Flash or PSRAM	Pre-computed, never modified
Envelope coefficient tables	Flash or PSRAM	Pre-computed by Builder
Tuning tables	Flash or PSRAM	Pre-computed by Builder
Firmware code	Flash (IRAM cached)	Never changes

Mutable Data

The following data changes during playback:

Data	Location	Reason
Voice Pool state	SRAM	Changes on every Note-On/Off and DMA interrupt
DMA audio buffers	SRAM	Written by synthesis, read by hardware
Mixer accumulation buffer	SRAM	Filled and cleared each DMA callback
MIDI state (pitch wheel, CC values)	SRAM	Changes on MIDI events

Data	Location	Reason
Per-voice per-chunk arrays	SRAM (in voice struct)	Computed each audio block

Embedded Notes — Memory Architecture

> **For ESP32:** The internal SRAM is the most precious resource. 320 KB sounds large but the system stack, WiFi (if enabled), and FreeRTOS take 80–100 KB immediately. Keeping the Voice Pool in internal SRAM ensures the DSP inner loop never stalls on an SPI bus.
> If PSRAM is available (ESP32-S3 with SPIRAM), the sample data can live there. If not, samples must stream from internal flash via the XIP (Execute In Place) cache — this works but adds latency variability.
> Recommendation: use ESP32-S3 with 8MB PSRAM for a production harmonium. The extra RAM eliminates all streaming concerns.

---

7. Sample Storage Strategy

Options Compared

Option A: Internal Flash (XIP)

Mechanism: Samples stored in the firmware flash partition, read via the ESP32's MMU/cache as if they were in address space. The XIP cache makes the first access slow (~microseconds), subsequent accesses fast.

Advantages:

- No external hardware needed
- Single firmware image (samples + code together)
- Transparent to the application (pointer read)
- Cache means hot samples are essentially free

Disadvantages:

- Internal flash is limited to 4–16 MB
- A full harmonium with 44 chromatic reed samples at 16-bit 44100Hz needs ~10–30 MB
- Flash write endurance (not relevant for read-only samples, but a concern for OTA updates)

Verdict: Suitable for small instruments (< 4 MB sample data) or lower sample rates.

Option B: External SPI Flash (W25Q128 or similar)

Mechanism: A separate 16–128 MB SPI flash chip connected to the ESP32's SPI bus. Accessed via `spi_flash_read()` or a mapped memory interface.

Advantages:

- Cheap, available in large sizes (up to 128 MB)
- Single PCB, no moving parts
- Can store multiple instruments
- Fast enough for sequential streaming (80 MHz SPI = 10 MB/s max)

Disadvantages:

- SPI bus contention (WiFi/BT also use SPI flash on ESP32)
- Read latency: 1–5 µs per transaction start
- No cache (unless mapped to the MMU)
- Single-ended SPI: max reliable rate ~5 MB/s for streaming

Verdict: Good for medium instruments when PSRAM is not available. Requires careful DMA prefetching to avoid read latency affecting audio.

Option C: SD Card

Mechanism: FAT32 filesystem on SD card, accessed via SDMMC or SPI interface.

Advantages:

- Enormous capacity (32 GB+)
- Easy for the user to update instruments (copy files via PC)
- Standard format

Disadvantages:

- Very high and unpredictable read latency (milliseconds for SD card wakeup)
- Filesystem overhead
- Power consumption
- Mechanical failure risk

- Cannot guarantee real-time streaming for audio

Verdict: Suitable only as a secondary storage for loading instruments at boot. Never stream samples from SD card in real-time.

Option D: PSRAM (External SPI RAM)

Mechanism: External pseudo-SRAM (SPIRAM) connected via the ESP32-S3's PSRAM interface. Accessed transparently through the CPU cache.

Advantages:

- Much faster than SPI flash (80 MHz in OPI mode = 40+ MB/s)
- Byte-addressable like RAM
- No read latency variability (cache-assisted)
- Can preload all samples at boot for true random access

Disadvantages:

- Limited to 4–8 MB on current ESP32 modules
- More expensive than external flash
- Slightly higher power than SRAM

Verdict: The optimal solution if available. Preload the entire sample dataset from flash into PSRAM at boot. From that point, all sample reads are cache-backed memory reads — extremely fast and deterministic.

Recommended Strategy

```
Boot Sequence:
  1. Read VHL package header from internal/external flash
  2. Validate checksum
  3. Copy Region Table, Sample Index, Lookup Tables → SRAM
  4. If PSRAM available:
       Copy all sample PCM data → PSRAM
  5. Else:
       Keep samples in flash, use DMA prefetch buffering
  6. Initialize Voice Pool
  7. Start I²S DMA
  8. Begin accepting MIDI
```

Primary recommendation: External flash (8–128 MB) for sample storage + PSRAM (8 MB) for runtime preloading. This combination gives large capacity and fast runtime access.

Fallback: Internal flash only, with lower sample rates (22050 Hz) and fewer samples to fit within the capacity limit.

Embedded Notes — Sample Storage

> **For ESP32:** The sample streaming architecture is the single biggest determinism risk. If a sample read blocks for 2 ms while the SPI bus resolves a transaction, that is a guaranteed audio glitch.
> The safest architecture: preload everything into PSRAM at boot. Then all sample reads are simple `memcpy`-style pointer reads through the cache — completely deterministic.
> If PSRAM is not available: use a double-buffered prefetch ring. The synthesis engine reads from a prefetch buffer while the DMA peripheral refills the other half of the buffer from flash. This is complex but necessary for flash-only configurations.

---

8. Runtime Data Structures

Design Principles

1. **No pointers into variable-length structures.** All runtime data is accessed by index into pre-allocated arrays.
2. **Fixed-size structures.** Every structure has a compile-time known size. No linked lists, no dynamic arrays.
3. **Cache-friendly layout.** Fields accessed together in the DSP loop are adjacent in memory.
4. **No inheritance.** Plain data structures (C-style structs conceptually). Virtual dispatch is too expensive.

Structure Hierarchy

```
VhlRuntime
    |
    ├─ VhlInstrumentTable[MAX_INSTRUMENTS]    (from VHL Manifest)
    |
    ├─ VhlRegionTable[MAX_REGIONS]            (from VHL Region Table)
    |
```

```
    ├─ VhlSampleTable[MAX_SAMPLES]          (from VHL Sample Index)
    │
    ├─ VhlVoicePool[MAX_VOICES]             (dynamic state, always in
SRAM)
    │
    └─ VhlMixer                             (stateless accumulator)
```

VhlInstrument

Represents one instrument (one preset) loaded from the manifest.

Field	Type	Description
instrument_id	uint8_t	Index into instrument table
bank, program	uint8_t	MIDI bank + program number
region_offset	uint16_t	First index into the Region Table
region_count	uint16_t	How many regions this instrument has
name	char[16]	Short display name

Ownership: immutable, loaded at boot into SRAM.

VhlRegion

One completely pre-baked zone. This is the central structure — it contains everything the Runtime needs to play one sample in response to a Note-On event.

Field Group	Fields
Range	key_lo, key_hi, vel_lo, vel_hi (uint8_t × 4)
Identity	exclusive_class, loop_mode, sample_id
Tuning	coarse_tune, fine_tune, scale_tuning
Volume	attenuation, pan, reverb_send, chorus_send
Volume Envelope	vol_delay, vol_attack, vol_hold, vol_decay, vol_sustain, vol_release
Key Scaling	vol_hold_scale, vol_decay_scale
Modulation Envelope	mod_delay, mod_attack, mod_hold, mod_decay, mod_sustain, mod_release
Mod→Pitch	mod_env_to_pitch (cents)
Mod→Filter	mod_env_to_filter_fc (cents)
Filter	filter_fc (Hz), filter_q

Field Group	Fields
Mod LFO	lfo_mod_delay, lfo_mod_freq, lfo_mod_to_pitch, lfo_mod_to_filter, lfo_mod_to_volume
Vib LFO	lfo_vib_delay, lfo_vib_freq, lfo_vib_to_pitch

Ownership: immutable, loaded at boot into SRAM. Never modified during playback.

VhlSample

Metadata for one sample.

Field	Type	Description
data_ptr	const int16_t *	Pointer to PCM data (in PSRAM or flash)
total_frames	uint32_t	Total PCM frame count
loop_start	uint32_t	Loop start frame
loop_end	uint32_t	Loop end frame
sample_rate	uint32_t	Source sample rate in Hz
root_key	uint8_t	MIDI root note
fine_tune	int8_t	Fine tune in cents

Ownership: immutable, loaded at boot into SRAM (metadata only). The `data_ptr` points to PSRAM or flash.

VhlVoice

One playing note. This is the only mutable per-note structure. Designed for cache-friendly DSP access.

Identity:

- region: const VhlRegion * (pointer to the immutable region)
- sample: const VhlSample * (pointer to the immutable sample)
- channel, key, vel (uint8_t × 3)
- active, release, finished (bool flags)

Playback state:

- current_pos: uint32_t (integer sample position)
- frac_pos: uint32_t (11-bit fixed-point fractional position)

- more_available: int32_t (samples to boundary)
- loop_mode: uint8_t (copied from region at voice start, may differ if modified)

Volume:

- volume_coeff: float (current smoothed volume coefficient)
- gain: float (master gain applied at creation time)

Filter state:

- x1, x2, y1, y2: float (biquad delay lines — survive across blocks)

Envelope state (per envelope: vol + mod):

- phase: uint8_t (delay/attack/hold/decay/sustain/quickAttack/release/off)
- chunk_counter: uint32_t (chunks elapsed in current phase)
- last_value: float (last computed envelope value)
- quick_release: bool

LFO state (per LFO: mod + vib):

- delay_done: bool
- delay_elapsed: uint32_t
- delta: float (current per-chunk increment, sign encodes direction)
- previous_point: float (last LFO output)
- previous_freq: float (last frequency, for smooth transition)

Per-chunk computation buffers:

- vol_array[N_CHUNKS]: float (one value per chunk)
- mod_array[N_CHUNKS]: float
- mod_lfo_array[N_CHUNKS]: float
- vib_lfo_array[N_CHUNKS]: float

Resampling scratch:

- src_scratch[8]: int16_t (boundary crossing scratch buffer)

Ownership: the Voice Pool owns all VhlVoice objects permanently. Voices are never created or destroyed — they are initialized (activated) and deactivated.

VhlMixer

Stateless per-block accumulator.

Field	Type	Description
left_buffer[BLOCK_SIZE]	float	Left channel accumulation
right_buffer[BLOCK_SIZE]	float	Right channel accumulation
dma_buffer[BLOCK_SIZE × 2]	int16_t	Final stereo I²S output

Ownership: global singleton, initialized once at boot.

Embedded Notes — Data Structures

> **For ESP32:** The critical insight is that `VhlVoice` is the hot path — it is accessed on every DMA interrupt. It must be in internal SRAM, not PSRAM. The per-chunk arrays inside VhlVoice are the largest per-voice cost. Reducing `N_CHUNKS` (i.e., reducing block size) reduces these arrays but increases interrupt frequency. A block size of 512 samples with 256-sample chunks means 2 chunk values per DMA block — the arrays are tiny.
> Cache alignment: place the DSP-critical fields (volume_coeff, filter state, envelope state) at the beginning of the struct so they fit in a single cache line.

---

9. Lookup Tables

Why Lookup Tables Are Central

The ESP32-S3's Xtensa LX7 core can execute integer operations in one cycle but floating-point transcendental functions (`sin` , `cos` , `exp` , `pow`) take dozens to hundreds of cycles. Every `pow()` or `log()` call in the DSP path is a potential performance bottleneck.

Lookup tables replace expensive runtime computations with fast indexed reads.

Table 1: Key × Velocity → Region List (Runtime Lookup)

Purpose: Given a Note-On with (key, vel), find all matching regions immediately.

Type: Builder-generated, loaded into SRAM at boot.

Structure:

```
For each instrument:
  region_list[region_count]: sorted array of VhlRegion indices
  key_first_index[128]: for each key, the first index in region_list
where key_lo ≤ key
```

At Note-On: Scan `region_list` from `key_first_index[key]` forward while `key_lo ≤ key`. Within each candidate, check `vel_lo ≤ vel ≤ vel_hi`. This is a short linear scan over a pre-sorted list, O(M) where M is typically 2–5 for a harmonium.

Alternative: A 2D 128×32-velocity-bucket table for O(1) lookup. Recommended when region count is large (drum kits with 128 velocity layers).

Table 2: Tuning Table (Builder-generated)

Purpose: Compute the resampling ratio from (played_key, root_key, fine_tune) without `pow2()` at runtime.

Structure:

```
tuning_ratio_q1616[128]:  Q16.16 fixed-point
  ratio = 2^(semitone_offset / 12)
  for semitone_offset in -64..+63
```

At Note-On / per chunk:

```
offset = played_key - root_key + coarse_tune
ratio = tuning_ratio_q1616[offset + 64]  // O(1) lookup
// Apply fine_tune: multiply by (1 + fine_cents/1200) - small
correction, linear approx OK
```

Table size: 128 entries × 4 bytes = **512 bytes** — negligible.

Table 3: Envelope Coefficient Table (Builder-generated)

Purpose: Compute the exponential decay/release coefficient without `pow()` at runtime.

Structure:

```
// Volume envelope decay: coef = 0.00001585^(1/timeChunks)
// Keyed by time in chunks (0–10000 ms at 172 chunks/s = 0–1720 chunks)
vol_decay_coef[1720]: float32
  vol_decay_coef[t] = pow(0.00001585, 1.0 / max(t, 1))
```

At envelope phase start: `decay_coef = vol_decay_coef[time_in_chunks]` — O(1).

Table size: 1720 × 4 bytes = **6.9 KB** per table. Two tables (decay + release) = ~14 KB.

Optimization: Use logarithmic quantization of time values to reduce the table to 256 entries with a small interpolation error.

Table 4: Sinc Filter Table (Optional, Builder-generated)

Purpose: 8-tap polyphase sinc resampling filter coefficients.

Structure:

```
sinc_table[256][8]: float32
  // 256 fractional phases × 8 tap coefficients
```

Table size: 256 × 8 × 4 = **8 KB** per window variant. One window = 8 KB. Keep in PSRAM if available.

For VHL v1 with linear interpolation: this table is not needed.

Table 5: Volume → Linear Gain Table (Optional)

Purpose: Convert MIDI CC7 volume (0–127) to a linear gain multiplier without runtime `pow10()`.

Structure:

```
cc7_to_gain[128]: float32
  cc7_to_gain[v] = pow(10, -0.05 * attenuation_from_cc7(v))
```

Table size: 128 × 4 = **512 bytes** — negligible.

Table 6: Pan → L/R Coefficient Table

Purpose: Equal-power panning without `cos()` / `sin()` at runtime.

Structure:

```
pan_coef_L[256]: float32   // cos(pan * pi/2 / 500) for pan -500..+500
quantized to 256 steps
pan_coef_R[256]: float32   // sin( ... )
```

Table size: 256 × 4 × 2 = **2 KB** — negligible.

Summary of Builder-Generated Tables

Table	Size	When Used
Key→Region index	~1 KB per instrument	Note-On
Tuning ratios	512 bytes	Note-On, per chunk
Vol envelope decay coeff	7 KB	Envelope phase start
Vol envelope release coeff	7 KB	Envelope phase start
Mod envelope coefficients	14 KB	Envelope phase start
Sinc filter (optional v2)	8 KB	Per sample (resampling)
CC7→gain	512 bytes	CC7 change
Pan L/R	2 KB	Note-On
Total	**~40 KB**	

40 KB of lookup tables fits comfortably in SRAM.

Embedded Notes — Lookup Tables

> **For ESP32:** The tuning table and envelope coefficient table are the two highest-value investments. They eliminate the two most expensive transcendental calls (`pow2` and `pow`) from the per-note-on and per-chunk path respectively.
> The pan table is a small but clean optimization. The CC7 table similarly.
> Total table cost: 40 KB in SRAM. Completely acceptable given the 168 KB headroom.

---

10. Voice Architecture

Static Voice Pool

All voices are pre-allocated at boot into a fixed-size array:

```
VhlVoice voice_pool[MAX_VOICES]    // e.g., MAX_VOICES = 32
bool     voice_active[MAX_VOICES]  // or use a field inside VhlVoice
```

No `malloc` . No `free` . No constructor/destructor during playback. Voices are activated (populated with region data) and deactivated (flagged as inactive). The memory is never returned to a heap.

Maximum Polyphony

Recommended default: 16 voices for a harmonium.

Why 16?

- A harmonium has at most one reed per key
- A 12-key chord uses 12 voices
- Reserve 4 for release tails
- 16 covers all realistic harmonium playing scenarios

Maximum possible: At 240 MHz, with a 512-sample DMA buffer:

- DMA interrupt fires every 512/44100 ≈ 11.6 ms
- With linear interpolation and simple float DSP, one voice costs roughly 0.5–1 ms of CPU
- Safe maximum: ~10–15 voices (leaves headroom for other system tasks)
- With optimization (fixed-point inner loop): potentially 24–32 voices

Voice Allocation Strategy

First-pass allocation:

1. Scan voice pool for a slot where `voice_active == false`
2. If found: activate that slot and return it
3. If not found: run voice stealing

Voice stealing (priority order):

1. Steal any voice where `voice_finished == true` (envelope at off phase) — no audible impact
2. Steal any voice where `voice_release == true` (in release phase) — minimal audible impact, steal quietest
3. Steal the oldest running voice with the lowest volume — most audible impact, last resort

Why this order: Stealing a finished or releasing voice causes no perceptible artifact. Stealing an active sustained voice causes an audible pop or cut, so it is the absolute last resort.

Exclusive Class (Voice Choke)

From Module 2 (voicelist.cpp), the exclusive class mechanism stops a previous voice when a new voice of the same class starts.

In the VHL Runtime, this is straightforward:

At Note-On, after allocating a voice:

```
if (new_region.exclusive_class ≠ 0) {
    for each active voice V:
        if V.exclusive_class == new_region.exclusive_class
           AND V.instrument_id == new_voice.instrument_id
           AND V.channel == new_voice.channel
           AND V.key ≠ new_voice.key:
            trigger_quick_release(V)
}
```

The quick release sets the voice's volume envelope to a 40ms release and marks it as releasing. The audio will fade out naturally within one or two DMA blocks.

Why this is important for harmonium: A harmonium stop mechanism (register change) may need to choke all current notes of a given register and restart them with different timbre. The exclusive class mechanism provides this capability.

Voice Lifecycle

```
INACTIVE → [Note-On, region match] → ACTIVE (Attack/Hold/Decay)
ACTIVE   → [Note-Off]              → RELEASING
ACTIVE   → [Exclusive class]       → QUICK-RELEASING
RELEASING → [Envelope = Off]       → FINISHED → INACTIVE (returned to
pool)
QUICK-RELEASING → [40ms elapsed]   → INACTIVE
ACTIVE   → [Sample end, no loop]   → FINISHED → INACTIVE
```

The transition from FINISHED → INACTIVE is performed by the Voice Pool scanner at the next DMA callback (not immediately, to avoid modifying the active list during DSP processing).

Voice Reuse Without Reset

When a voice is reactivated for a new note, only the fields that change between notes need to be reset:

- Playback position → reset to sample start
- Envelope state → reset to Delay/Attack phase
- LFO state → reset to 0 + delay
- Filter delay lines → reset to 0 (to prevent contamination from previous note)
- Volume coefficient → reset to 0

This is faster than a `memset` on the entire struct.

Embedded Notes — Voice Architecture

> **For ESP32:** 16 voices with a static pool is the correct starting point. The ESP32 has sufficient RAM and CPU for 16 voices with linear interpolation and float32 DSP.
> Voice stealing from the release phase is the critical safety net. Without it, a fast arpeggio on a harmonium would exhaust the voice pool and cause the oldest notes to cut out without a release.
> The exclusive class mechanism doubles as a "stop" mechanism for future register-switching functionality. It should be implemented even in v1.

---

11. Timing Architecture

The Four Time Domains

```
┌──────────────────────────────────────────────────────────────────────┐
│ DOMAIN 1: LOAD TIME (occurs once at boot)                            │
│                                                                      │
│                                                                      │
│ — Read VHL file header and validate                                  │
│                                                                      │
│ — Copy Region Table → SRAM          (O(N_regions))                   │
│ — Copy Sample Index → SRAM          (O(N_samples))                   │
│ — Copy Lookup Tables → SRAM         (O(table_sizes))                 │
│ — Preload sample data → PSRAM       (O(total_sample_bytes))          │
│ — Zero-initialize Voice Pool        (O(MAX_VOICES))                  │
│ — Configure I²S DMA                 (O(1))                           │
│                                                                      │
│ Duration: 100ms – 2s depending on sample data size                   │
│ Constraint: none (user waits for boot)                               │
│ Operations: may use division, sqrt, memcpy, SPI reads                │
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│ DOMAIN 2: NOTE-ON / NOTE-OFF (occurs on MIDI events)                 │
│                                                                      │
```

```
| — Receive MIDI event                O(1)                                |
| — Resolve instrument                O(1) table lookup                   |
| — Scan Region Table for matches     O(N_regions) — small, fast          |
| — Apply velocity→attenuation        O(1) table lookup                   |
| — Allocate voices from pool         O(MAX_VOICES) worst case            |
| — Initialize voice state            O(1) per voice                      |
| — Set envelope to attack phase      O(1)                                |
| — Exclusive class check             O(MAX_VOICES) worst case            |
|                                                                         |
|                                                                         |
| Duration: < 20 µs target (< 50 µs acceptable)                           |
| Constraint: must complete before perceptible latency (~20ms)            |
|                                                                         |
| Operations: no pow(), no log(), no malloc() — only integer ops          |
|                                                                         |
└─────────────────────────────────────────────────────────────────────────┘


┌─────────────────────────────────────────────────────────────────────────┐
| DOMAIN 3: PER DMA BLOCK (occurs every ~11.6ms at 44100Hz, 512 smp)      |
|                                                                         |
|                                                                         |
| — DMA interrupt fires               (hardware trigger)                  |
| — Swap DMA buffers                  O(1) pointer swap                   |
| — Zero mixer accumulation buffers   O(BLOCK_SIZE)                       |
| — For each active voice:                                                |
|                                                                         |
|     — Voice::processBlock()         (detailed below)                    |
|     — Accumulate into mixer         O(BLOCK_SIZE)                       |
| — Hard clip L/R output              O(BLOCK_SIZE)                       |
| — Convert float → int16             O(BLOCK_SIZE)                       |
| — Write to DMA buffer               O(BLOCK_SIZE)                       |
| — Detect and collect finished voices O(MAX_VOICES)                      |
|                                                                         |
|                                                                         |
| Duration: must complete in < 11.6ms (the DMA buffer period)             |
|                                                                         |
| Constraint: hard real-time — if late, audio glitches                    |
| Operations: float32 arithmetic OK, no dynamic allocation, no I/O        |
|                                                                         |
└─────────────────────────────────────────────────────────────────────────┘


┌─────────────────────────────────────────────────────────────────────────┐
| DOMAIN 4: PER CHUNK (256 samples, inside Voice::processBlock)
```

```
│
│
│
│ — Volume Envelope step              O(1) state machine
│
│ — Modulation Envelope step          O(1) state machine
│
│ — Mod LFO step                      O(1) triangle oscillator          │
│ — Vib LFO step                      O(1) triangle oscillator          │
│ — Compute pitch delta (cents)       O(1) additions only               │
│ — Convert cents → ratio             O(1) table lookup                 │
│ — Compute effective filter freq     O(1) table lookup                 │
│ — Compute biquad coefficients       O(1) with fast sin/cos approx     │
│ — Compute volume deltaCoeff         O(1)                              │
│
│
│ Duration: < 1 µs per chunk (for 16 voices at 172 chunks/s: 2.7ms)    │
└──────────────────────────────────────────────────────────────────────────┘
┘

┌──────────────────────────────────────────────────────────────────────────┐
┐
│ DOMAIN 5: PER SAMPLE (innermost loop, 44100 iterations per second)
│
│
│
│ — Advance fractional position       O(1) add                          │
│ — Detect loop boundary              O(1) compare                      │
│ — Wrap loop position                O(1) subtract                     │
│ — Read 1–2 source samples           O(1) memory read                  │
│ — Linear interpolation              2 multiplications                 │
│ — Advance volume coefficient        1 addition                        │
│ — Apply biquad filter               5 mult + 4 add                    │
│ — Apply volume                      1 multiplication                  │
│
│
│ Per voice: ~10 multiplications + ~8 additions per sample
│
│ At 44100 Hz, 16 voices: 7 million operations per second              │
│ ESP32-S3 at 240 MHz: 240 million cycles/s → ~30 cycles/sample/voice  │
│ → approximately 480 cycles per output sample for 16 voices           │
│ → feasible, leaving ~50% CPU for other tasks                         │
└──────────────────────────────────────────────────────────────────────────┘
┘
```

Timing Budget Analysis

At 44100 Hz, 512-sample DMA buffer, 16 voices:

- DMA callback period: 11.6 ms
- Available CPU cycles: 240 MHz × 11.6 ms = 2.78 billion cycles
- Per-sample inner loop (16 voices): ~480 cycles × 512 = 246K cycles
- Per-chunk DSP (16 voices, 2 chunks per block): ~1000 cycles × 2 × 16 = 32K cycles
- Total DSP estimate: ~280K cycles out of 2.78M available = **~10% CPU**

This is excellent. It means:

- 90% CPU is available for MIDI, display, WiFi, BT
- Voice count could increase to 40+ without hitting the limit
- Quality upgrades (sinc interpolation, reverb) are feasible in later versions

Embedded Notes — Timing

> **For ESP32:** The timing analysis confirms that 16 voices with linear interpolation is a very conservative target. The real bottleneck will likely be PSRAM latency (if streaming sample data) rather than CPU.
> The DMA callback timing is the hardest constraint. The MIDI event handling must never be allowed to disable interrupts for more than a few microseconds. Use a simple ring buffer for MIDI input that the DMA callback processes.

---

12. CPU Optimization Strategy

Strategy 1: Per-Chunk Amortization

The most impactful optimization. Envelope values, LFO values, filter coefficients, and pitch ratios change slowly (at audio rates below 100 Hz). They need not be computed per sample.

Compute them once per chunk (256 samples):

- 256× reduction in envelope evaluation cost
- 256× reduction in LFO evaluation cost
- 256× reduction in filter coefficient computation cost

Within a chunk, use the single computed value for all 256 samples. For volume, use linear interpolation between chunks (as Polyphone does) to prevent zipper noise.

Strategy 2: Lookup Table Replacement of Transcendentals

All `pow()`, `log()`, `exp()`, `sin()`, `cos()` calls must be replaced with:

- Precomputed tables (for fixed-frequency operations like tuning)
- Polynomial approximations (for modulated operations like biquad sin/cos)
- Fixed-point approximations (for volume conversions)

Specific replacements:

- `pow2(cents/1200)` → tuning table (O(1) lookup)
- `pow(decay_coeff, 1/time)` → envelope coefficient table (O(1) lookup)
- `sin(theta)`, `cos(theta)` for biquad → minimax polynomial approximation (3–5 terms)
- `pow10(gain_dB/20)` → small table for CC7 levels

Strategy 3: Fixed-Point Arithmetic for the Inner Loop

The sinc/linear interpolation inner loop and the volume scaling loop are the hottest code. They run 44,100 times per second per voice.

Fixed-point representation for the resampling cursor:

```
uint32_t fixed_pos = (integer_pos << 16) | fractional_pos
```

- Upper 16 bits: integer sample position
- Lower 16 bits: fractional position (0–65535)
- Advancing: `fixed_pos += pitch_ratio_q16`
- Extracting fraction for interpolation: `frac = fixed_pos & 0xFFFF`
- Extracting integer offset: `offset = fixed_pos >> 16`

This eliminates all floating-point from the inner sample-read loop.

Strategy 4: Branch Elimination

Branches inside the per-sample loop are expensive because they break the CPU pipeline.

The biquad filter bypass (when filter is fully open) is handled by:

- Selecting either the filter loop or the non-filter loop **once per chunk** based on whether filtering is active
- Using a function pointer or conditional to select the path before the loop starts

- Inside the loop: no branches

The loop mode check (looping vs non-looping) is similarly resolved:

- Compute `is_looping` once per chunk
- Select the loop boundary check variant before entering the per-sample loop

Strategy 5: Cache Locality

The per-voice DSP reads the voice struct repeatedly. Ensuring the most frequently accessed fields (volume_coeff, filter state, envelope last_value, playback position) are at the start of the struct ensures they fit in one or two cache lines.

For sample data:

- Sequential playback reads are cache-friendly by nature
- Loop wrap (going back to loopStart) may cause a cache miss
- Mitigation: at loop wrap time, the next RESAMPLING_ORDER samples can be prefetched

Strategy 6: SIMD / DSP Instructions

The Xtensa LX7 core (ESP32-S3) has:

- FPU with hardware single-precision float operations
- No SIMD, but multiply-accumulate in hardware
- Result: float32 DSP is fast — use it for the inner loop

The ESP32-S3 also has a dedicated **DSP acceleration unit** (if ESP32-S3 variant supports it). This can accelerate 16-bit MAC operations. For a Q15 fixed-point inner loop, this could triple throughput.

Strategy 7: DMA Double-Buffering

While the DMA peripheral is transmitting one buffer, the CPU fills the other. This ensures the CPU never has to wait for the DMA — they run truly in parallel. The DMA interrupt fires to signal the swap, which is the only synchronization point.

Embedded Notes — CPU Optimization

> **For ESP32:** The per-chunk amortization is the single highest-leverage optimization. It costs no accuracy (the 256-sample period is musically inaudible) and reduces DSP overhead by ~256×.
> The fixed-point inner loop is important but only necessary if the float version

> cannot sustain 16+ voices. Profile first before optimizing.
> Lookup tables for transcendentals should be implemented from the start — they are simple to implement and provide guaranteed performance.

---

13. Scalability

Adding New Instruments

The architecture scales to multiple instruments trivially:

- Add new entries to the Instrument Manifest section
- Add new regions to the Region Table
- Add new samples to the Sample Data section
- Regenerate the Lookup Tables for new instruments
- All regions and samples are globally indexed — no per-instrument addressing change needed

The Runtime selects the correct instrument based on MIDI bank + program. No code change is needed to add a new instrument.

Drum Kits

Drum kits are instruments where each key plays a different sample, typically with no looping and a short release.

In the VHL format, a drum kit is just another instrument with:

- 128 regions, each with a different key range (usually one key per region)
- loop_mode = 0 (no loop)
- Short release time
- exclusive_class values for cymbal choke groups

No special casing is needed. The Builder and Runtime handle drum kits identically to melodic instruments.

Layered Instruments

Multiple regions matching the same key × velocity range produces simultaneous voices (as in Polyphone). This is the correct SF2 behavior and is supported natively.

A piano layer (bright + warm samples summed) is just two regions with identical key/vel ranges but different sample_ids and attenuation values.

Velocity Layers

Multiple non-overlapping velocity ranges on the same key produce different samples at different dynamics:

- pp sample: vel_lo = 0, vel_hi = 48
- mp sample: vel_lo = 49, vel_hi = 96
- ff sample: vel_lo = 97, vel_hi = 127

The VHL lookup table (key × velocity range scan) handles this correctly without special cases.

Velocity Crossfading

Softer implementation: assign slightly overlapping velocity ranges and use the velocity→attenuation modulator (default modulator, baked at Builder time) to fade between layers. The two overlapping voices are both triggered, but the quieter one fades out according to its velocity.

Future DSP Effects

The architecture supports a future DSP chain by adding an effects stage after the Mixer:

```
[Mixer Output] → [Effects Chain] → [I²S DMA]
```

The Effects Chain can include:

- Simple delay/reverb (if sufficient RAM)
- Chorus (requires a circular delay buffer)
- Simple EQ (biquad chains, identical to the voice filter)

The key constraint: effects must be DSP-budget-compatible. With ~90% CPU headroom, there is substantial room for effects in future versions.

Multiple Presets Per Instrument

A VHL preset is an instrument entry in the manifest. Multiple presets (e.g., "Harmonium Loud", "Harmonium Soft") can reference the same underlying sample set but with different region parameters (different attenuation, different envelope times).

This is handled by adding multiple instrument entries in the manifest that share sample data but have different region tables. No change to the Runtime.

Embedded Notes — Scalability

> **For ESP32:** The architecture is inherently scalable because all variability is captured in the VHL file. Adding a new instrument means regenerating the VHL file and reflashing — the Runtime firmware never changes.
> This is the correct embedded philosophy: the data (VHL) is variable, the code (Runtime) is stable.

---

14. Future Hardware Expansion

Design Principle for Hardware Expansion

The Runtime should use hardware abstraction layers (HAL) for all I/O. When a new hardware component is added, only the HAL layer changes. The DSP engine, voice management, and MIDI handling remain untouched.

I²S DAC

Current: Internal I²S with a PCM5102 or similar external DAC.

Future expansion:

- Higher bit depth (24-bit): requires changing the I²S peripheral configuration only
- Higher sample rate (96 kHz): requires rebuilding the VHL package at 96 kHz + recalculating DSP budgets
- Different DAC chip: change the I²S init sequence only

The Runtime's audio output section is a single function `write_dma_buffer(int16_t *buf, size_t len)`. Changing the DAC means changing only this function.

MIDI Input

Current: UART MIDI at 31.25 kbps.

Future expansion:

- USB MIDI (via ESP32-S2/S3 USB peripheral): requires a USB MIDI class driver. The MIDI event processing code inside the Runtime is identical.
- Bluetooth MIDI (BLE-MIDI): requires a BLE stack. Again, the event processing is identical.
- TRS-MIDI Type A/B: different connector, same UART hardware.

The MIDI event handler is decoupled from the transport: `process_midi_event(channel, type, data1, data2)` is the only interface. The transport layer feeds events into this function regardless of whether they came from UART, USB, or BLE.

OLED Display

An OLED display showing instrument name, note activity, or tuning information can be added without affecting the audio engine.

The display update runs on a separate FreeRTOS task at a low priority (10–30 Hz). It reads voice state from the Voice Pool (read-only, no locks needed for display) and renders accordingly.

The critical constraint: the display task must never preempt the DMA callback. Since the DMA callback runs at the highest interrupt priority, this is guaranteed by the RTOS.

SD Card

The SD card can serve as:

- Primary instrument storage (boot from SD, load into PSRAM)
- Instrument library (select instruments from a menu)
- Recording output (not needed for harmonium, but useful for future)

SD card access is only performed outside of the audio path. At boot: load VHL from SD into PSRAM. During playback: no SD access.

Effects Processor (External DSP)

For high-quality reverb or chorus beyond the ESP32's capability, an external DSP chip (e.g., FV-1 reverb chip, or a second ESP32 dedicated to effects) can be added.

Integration: the I²S output of the main ESP32 is routed through the external DSP before the DAC. No change to the Runtime is needed. The Main ESP32 "does not know" about the effect.

Embedded Notes — Hardware Expansion

> **For ESP32:** Design the HAL interfaces now, even if only one implementation exists at first. A `midi_input_read_event()` abstraction, an `audio_output_write_buffer()` abstraction, and a `display_show_state()` abstraction ensure that hardware changes never touch the DSP engine.

> The greatest risk to long-term maintainability is coupling the DSP code to a specific hardware peripheral. Avoid this from the beginning.

---

15. Source Asset Mapping

From Module 1 and Module 2 to VHL Architecture

The following table maps every major algorithm and data structure from the Polyphone study (Modules 1 and 2) to its destination in the VHL system:

Polyphone Component	VHL Stage	Notes
SF2 Parser (`InputParserSf`)	Builder	Parse SF2 binary format → internal model
SFZ Parser (`InputParserSfz`)	Builder	Parse SFZ text format → internal model
SoundFontManager	Builder + VHL Format	Hierarchy management → flattened Region Table
SF2 bag-based zone discovery	Builder	Identifies global vs zone parameters → pre-baked into regions
SFZ global→group→region push-down	Builder	SFZ inheritance → flat regions
VoiceParam initialization	Builder (partial) + Runtime	Parameter merging → baked. MIDI-dynamic params → runtime
ModulatorGroup + ParameterModulator	Builder (partially) + Runtime	Default 10 modulators → 8 baked, 2 dynamic (velocity, pitch wheel)
ModulatedParameter accumulation	Builder	inst + prst parameter addition → single baked value per field
Attribute range limiting	Builder	All clamping done at build time
champ_startAddrsOffset computations	Builder	Absolute sample start/end/loop addresses pre-computed
Synth::playPrst() + playInst()	Runtime	Region lookup (simplified to flat table scan)

Polyphone Component	VHL Stage	Notes
VoiceList ring buffer	Runtime	Replaced by simpler flat Voice Pool array
VoiceList::prepareComputation()	Runtime	Voice stealing + exclusive class — kept
Voice::initialize()	Runtime	Voice state reset on Note-On
Voice::generateData()	Runtime	Per-block DSP — core of the Runtime
Pitch formula (`deltaPitchFixed` **)**	Runtime (simplified)	Table lookup replaces ln() computation
Fixed-point resampling cursor	Runtime	Kept exactly — fundamental design
`getData()` / `getDataWithLoop()`	Runtime	Loop logic — kept exactly
Sinc interpolation (8-tap)	Runtime (v2)	Replaced by linear interpolation in v1
Sinc table generation	Builder (for v2)	Pre-computed by Builder
EnveloppeVol state machine	Runtime	Kept exactly — 8 states, per-chunk evaluation
Envelope time→coefficient pow()	Builder	Pre-computed envelope coefficient table
Key scaling for hold/decay	Runtime	Kept: `time_hold *= 2^(noteToHold * (60 - note))` simplified
OscTriangle LFO	Runtime	Kept exactly — triangle oscillator, per-chunk
biQuadCoefficients()	Runtime	Kept — simplified with polynomial sin/cos approx
Volume coefficient interpolation	Runtime	Kept exactly — prevents zipper noise
SoundEngine::generateData() mixer	Runtime	Simplified: no 8-bus routing, just L+R
Equal-power pan law	Runtime	Kept — table lookup for cos/sin
FreeVerb reverb	Not in v1	Future: add after basic engine is stable
STK Chorus	Not in v1	Future

Polyphone Component	VHL Stage	Notes
Synth thread architecture	Eliminated	Single DMA callback thread replaces N-thread model
Qt signal/slot system	Eliminated	Not applicable to embedded
QMutex synchronization	Eliminated	Single-threaded DMA callback needs no mutex
Recorder	Not in v1	Future: SD card recording
LiveEQ	Not in v1	Future

Builder Asset Summary

Asset	From Polyphone	Transformed To
SF2/SFZ parser	InputParserSf + InputParserSfz	Builder's parsing module
Zone hierarchy resolution	VoiceParam::readDivisionAttributes	Builder's region flattening
Modulator pre-evaluation	ModulatorGroup + ParameterModulator	Builder's default-mod baking
Sample address computation	VoiceParam::getPosition (offsets)	Builder's absolute address computation
Envelope coefficient generation	EnveloppeVol formulas (pow calls)	Builder's lookup table generator
Tuning table generation	Voice pitch formula	Builder's pitch ratio table
Region table packaging	VoiceParam struct	Builder's binary packing
Sinc table generation (v2)	Voice::prepareTables() + fillSincTable	Builder's DSP table generator

Runtime Asset Summary

Asset	From Polyphone	Kept or Modified
Region scan (Note-On)	Synth::playPrst + playInst	Kept, simplified to flat scan

Asset	From Polyphone	Kept or Modified
Voice pool	VoiceList	Simplified: flat array, no ring buffer
Voice activation	Voice::initialize	Kept
Playback cursor	`_currentSmplPos + _lastDistanceFraction`	Kept exactly
Loop wrap	Voice::getDataWithLoop()	Kept exactly
Envelope state machine	EnveloppeVol	Kept, 8 states
Triangle LFO	OscTriangle	Kept exactly
Per-chunk DSP cadence	COMPUTATION_CHUNK = 256 pattern	Kept exactly
Per-sample volume interp	deltaCoeff pattern	Kept exactly
Biquad filter	biQuadCoefficients + inner loop	Kept, approx sin/cos
Equal-power pan	SoundEngine pan law	Kept, table lookup
Hard clipping	FastMaths::clamp	Kept
Voice stealing	VoiceList::prepareComputation	Kept, simplified
Exclusive class choke	VoiceList exclusive class logic	Kept

---

16. Engineering Trade-offs

Trade-off 1: Builder Complexity vs Runtime Simplicity

Decision: Maximize Builder complexity to minimize Runtime complexity.

Problem solved: The ESP32 has strict CPU and RAM budgets. Every computation in the Runtime consumes real-time resources. Computations in the Builder are unlimited.

Alternatives considered:

- Lazy computation at runtime (compute what you need when you need it): causes Note-On latency spikes and unpredictable DSP load.
- Precompute only the most expensive operations: intermediate approach, harder to reason about which operations are "expensive enough."

Why this is appropriate: The Builder runs once. Its output (the VHL file) is validated and immutable. The Runtime is simple and therefore easy to debug, audit, and trust.

Trade-off 2: Flash Storage vs PSRAM Preloading

Decision: Use external SPI flash for primary storage, preload samples to PSRAM at boot.

Problem solved: Flash access is slow and non-deterministic for streaming. PSRAM provides fast, cache-backed access.

Alternatives considered:

- Stream directly from flash: works but causes occasional latency spikes due to SPI arbitration. Requires complex DMA prefetch logic.
- Store everything in internal SRAM: impossible — a harmonium with 44 reed samples at 44100Hz needs ~20 MB, far exceeding 320 KB internal SRAM.
- SD card streaming: SD card latency is too high and too variable for real-time audio.

Trade-off: PSRAM costs money and board space. But it eliminates the most complex part of the runtime (sample streaming logic) and provides deterministic behavior.

Trade-off 3: Polyphony (Voice Count) vs Audio Quality

Decision: 16 voices at 44100 Hz with linear interpolation.

Problem solved: Higher voice count requires more CPU. Better interpolation (sinc) costs more CPU per voice. The trade-off space is continuous.

Analysis (from Section 11 timing estimate):

- 16 voices at 44100 Hz: ~10% CPU
- 32 voices at 44100 Hz: ~20% CPU
- 16 voices with sinc: ~30% CPU (8× more per sample)
- 32 voices with sinc: ~60% CPU (risky for real-time safety)

Decision: Start with linear interpolation at 16 voices. Upgrade to sinc (for quality) or more voices (for complex chords) once the baseline is validated. The

architecture supports both upgrades without structural changes.

Trade-off 4: Generality vs Simplicity

Decision: Design for a harmonium first, generalize second.

Problem solved: A fully generic SF2 runtime (supporting all 60 generators, all modulators, all sample types, all loop modes) is complex and expensive. A harmonium-specific runtime that handles only what harmonium sounds need is simple, fast, and correct.

What generality was traded:

- Custom modulator chains (not used in harmonium)
- Release trigger mode (loop_mode 2) (not used in harmonium)
- 24-bit sample support (16-bit is sufficient)
- Full reverb/chorus (no reverb needed for harmonium)

What was preserved for future generality:

- The region table structure supports any key/vel mapping
- The voice pool supports any polyphony configuration
- The envelope state machine supports any DAHDSR timing
- The loop modes support 0, 1, and 3 (all used in harmonium)
- The exclusive class mechanism is generic

Trade-off 5: Single vs Multi-Core Audio

Decision: Single-core audio. The DMA callback runs on CPU0. CPU1 handles MIDI, display, BLE.

Problem solved: Multi-core DSP on ESP32 requires shared memory synchronization, which introduces non-determinism (locks, spinwaits) into the real-time audio path.

Alternative considered: Pin each voice to a CPU0 or CPU1 thread, use lock-free queue to merge outputs. This is the Polyphone model (N-thread SoundEngine). On ESP32, the overhead of cross-core signaling and cache coherence makes this more complex than the single-core version for 16 voices.

Why single core is correct for this scale: At 10% CPU for 16 voices, there is no need for multi-core DSP. Adding multi-core DSP at this scale would add complexity and potential bugs without any performance benefit.

Trade-off 6: Flat Regions vs Hierarchical Instrument Model

Decision: Fully flat regions. No hierarchy at runtime.

Problem solved: Hierarchy resolution at runtime requires pointer traversal, conditional logic for global zones, and merging algorithms. All of this costs time at Note-On and risks cache misses.

Alternative: Keep a lightweight hierarchy at runtime (instrument global zone + per-zone parameters). This saves some storage (fewer redundant parameter copies) at the cost of runtime merging.

Why flat wins: Storage is cheap. A region is ~100 bytes. 100 regions = 10 KB — trivial. Merging at runtime costs 100+ ns per Note-On for each merged field. The flat design gives O(1) parameter access for every field.

Trade-off 7: Fixed-Point vs Float32 Arithmetic

Decision: Float32 for DSP, fixed-point only for the resampling cursor.

Problem solved: Full fixed-point DSP is faster on platforms without an FPU but much harder to implement and debug. The Xtensa LX7 (ESP32-S3) has a hardware FPU that makes float32 arithmetic nearly as fast as fixed-point.

Where fixed-point is used:

- The resampling cursor (uint32_t with 16-bit fraction) — because it needs integer comparisons for loop boundary detection, and floating-point comparisons are less clean here.

Where float32 is used:

- Envelope values (range 0..1, needs precision)
- Filter coefficients and delay lines (requires precision for stability)
- Volume coefficients (range 0..2, interpolated)
- LFO output (range −1..+1)

Trade-off 8: DMA Buffer Size vs Latency

Decision: 512-sample DMA buffer (11.6 ms at 44100 Hz).

Problem solved: Smaller buffers reduce latency but increase interrupt rate (more CPU overhead). Larger buffers increase latency but reduce interrupt overhead.

Analysis:

- 128 samples: 2.9 ms latency, 344 interrupts/second — very tight DSP deadline
- 256 samples: 5.8 ms latency, 172 interrupts/second — feasible
- 512 samples: 11.6 ms latency, 86 interrupts/second — comfortable DSP budget
- 1024 samples: 23.2 ms latency, 43 interrupts/second — noticeable latency on a harmonium

Decision: 512 samples is the best compromise. 11.6 ms is below the perceptible audio latency threshold (~20–30 ms), and the interrupt rate is comfortable for the DSP budget.

End of Module 3 — Embedded Runtime Architecture & VHL System Design
This document is the architectural blueprint for all subsequent implementation work.

Appendix: Key Constants

Constant	Value	Rationale
DMA Block Size	512 samples	11.6 ms latency at 44100 Hz
Chunk Size	256 samples	Per-chunk DSP amortization, from Polyphone study
Max Voices	16 (v1) / 32 (v2)	10–20% CPU at 44100 Hz
Max Instruments	8 (v1) / 32 (v2)	VHL manifest limit
Max Regions per Instrument	256	uint8_t index, sufficient for all harmonium stops

Constant	Value	Rationale
Max Samples per Instrument	128	uint8_t index
Sample Rate	44100 Hz	CD quality, I²S standard
Bit Depth	16-bit signed	I²S standard, DAC standard
Resampling Cursor Fraction Bits	16	Q16.16 fixed-point
Envelope Coefficient Table Size	1720 entries	0–10 s at 172 chunks/s
Tuning Table Size	128 entries	−64 to +63 semitones
Exclusive Class Max	255	uint8_t in region
Quick Release Time	40 ms	From Polyphone: 0.04 × chunkRate

Next: Module 4 — VHL Builder Implementation
Next: Module 5 — ESP32 Runtime Implementation